

Wireless Drive-By-Wire

User Manual

Capstone Spring 2026

ECE 4773 — Senior Design

Document Version:	1.0
Date:	May 2026
Status:	Final

0 Contents

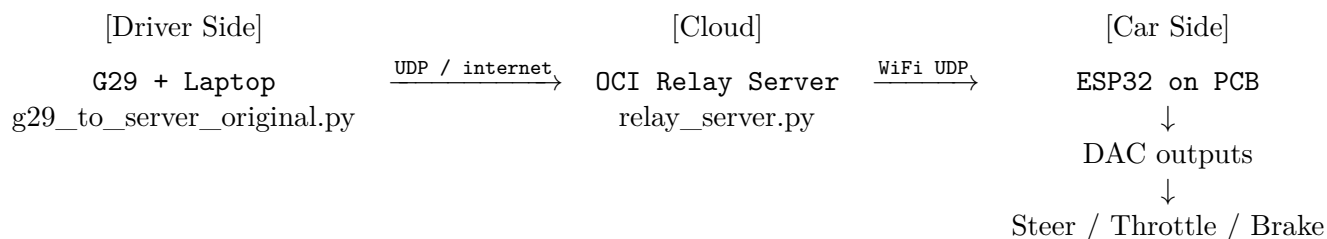
1	Introduction	2
1.1	System Architecture	2
2	Safety	2
3	Hardware Overview	3
3.1	Component List	3
3.2	ESP32 Pin Reference	3
3.3	DAC Channel Assignments	4
4	Software Installation	4
4.1	Operator Laptop	4
4.2	OCI Relay Server	4
4.3	ESP32 Firmware	4
5	Pre-Operation Checklist	5
6	Operating the System	5
6.1	Starting a Session	5
6.2	Control Mapping	6
6.3	Stopping a Session	6
7	How It Works	6
7.1	Step 1 — Car Registers with the Relay Server	6
7.2	Step 2 — Operator Sends Control Commands	6
7.3	Step 3 — Relay Server Forwards to the Car	7
7.4	Step 4 — ESP32 Drives the Actuators	7
8	Troubleshooting	7
9	Multi-Camera Streaming with OBS	8

1 Introduction

The Wireless Drive-By-Wire (DBW) system allows an operator to remotely steer, accelerate, and brake a Nissan Leaf using a Logitech G29 racing wheel and pedal set connected to a laptop anywhere with an internet connection.

Control inputs are captured at **60 Hz**, transmitted over UDP to a cloud relay server hosted on Oracle Cloud Infrastructure (OCI), and forwarded to an ESP32 microcontroller installed on a custom PCB inside the vehicle. The ESP32 runs a closed-loop PID controller for steering—reading the actual steering angle from the vehicle’s CAN bus—and drives open-loop throttle and brake actuators through dual MCP4728 digital-to-analog converters (DACs).

1.1 System Architecture



The relay server is required because the ESP32 is behind a mobile hotspot with no public IP. The ESP32 registers itself via periodic **HEARTBEAT** packets; the relay server records that public endpoint and uses it to forward incoming **CMD** packets from the operator (UDP hole-punching).

2 Safety

!! DANGER

- A trained safety driver **must** be seated and belted in the vehicle at all times.
- The safety driver must be ready to take manual control **immediately**.
- **Never** operate on public roads. Use only a closed, controlled area.
- **Never** exceed walking pace during initial testing.
- Keep all bystanders at least **50 feet** from the vehicle at all times.

△ WARNING

- If the operator laptop stops sending (Ctrl+C or network loss), the ESP32 has no watchdog — it will continue targeting the **last commanded** steering angle, throttle, and brake indefinitely. The safety driver must take manual control immediately if the connection is interrupted.
- Writing 0 V to the brake DAC channels is **not** a neutral state and may be interpreted as a fault by the vehicle ECU. The firmware always drives brake channels to their baseline voltages.

- Do not modify PID gains without re-validating on a stationary vehicle first.

3 Hardware Overview

3.1 Component List

Component	Qty	Notes
Nissan Leaf (2015–2018)	1	Vehicle under control; CAN ID 0x2 carries steering angle
Logitech G29 Wheel + Pedals	1	USB to operator laptop
Operator Laptop	1	Any Python 3.8+ capable machine
ESP32 DevKit V1	1	Mounted on custom DBW PCB inside vehicle
Custom DBW PCB	1	Houses ESP32, CAN controller, and both DACs
MCP2515 CAN Controller	1	On-board; reads steering angle via SPI
MCP4728 DAC ×2	2	On-board; drives steering, throttle, brake
Mobile Hotspot	1	Provides WiFi to the ESP32 in the vehicle
OCI Relay Server	1	Oracle Cloud VM; UDP port 4210 must be open

3.2 ESP32 Pin Reference

Signal	ESP32 Pin
SPI SCK	18
SPI MISO	19
SPI MOSI	23
CAN CS	5
CAN INT	4
I2C Bus 1 SDA / SCL (DAC 1)	21 / 22
I2C Bus 2 SDA / SCL (DAC 2)	33 / 32

3.3 DAC Channel Assignments

DAC	Ch	Function	Baseline
DAC 2	C	Steering main	2.47 V
DAC 2	D	Steering sub	2.47 V
DAC 1	A	Throttle main	0.37 V
DAC 1	B	Throttle sub	0.75 V
DAC 1	C	Brake main	3.42 V
DAC 1	D	Brake sub	1.51 V

Note

All DAC channels use the MCP4728 internal voltage reference (MCP4728_VREF_INTERNAL) with $2\times$ gain, giving a full-scale output of 4.096 V and 12-bit resolution ($\approx 1\text{ mV/step}$).

4 Software Installation

4.1 Operator Laptop

Install Python 3.8 or newer, then install the only required library:

```
pip install pygame
```

The server IP is already configured in `g29_to_server_original.py`.

4.2 OCI Relay Server

The relay server uses only the Python standard library:

```
python3 relay_server.py
```

Verify that UDP port **4210** is open in both the OCI Security List and the instance firewall (`iptables` / `firewalld`).

4.3 ESP32 Firmware

4.3.0 Required Tools

- Arduino IDE 2.x
- ESP32 board support package (install via **File** → **Preferences** → **Additional Board Manager URLs**, then search “esp32” in **Tools** → **Board Manager**)
- Library: **Adafruit MCP4728** (Tools → Manage Libraries)
- Library: **mcp_can** by Seeed Studio (Tools → Manage Libraries)

Note

WiFi and WiFiUdp ship with the ESP32 board package—do **not** install them separately.

4.3.0 Configure WiFi Credentials

Open `esp32_car.ino` and set the hotspot SSID and password:

```
const char* ssid      = "YOUR_HOTSPOT_NAME";  
const char* password = "YOUR_HOTSPOT_PASSWORD";
```

4.3.0 Flash the Firmware

1. Connect the ESP32 to a computer via USB.
2. **Tools** → **Board** → **ESP32 Dev Module**.
3. **Tools** → **Port** → select the correct port (COM3 on Windows, `/dev/ttyUSB0` on Linux).
4. Click **Upload** and wait for “Done uploading.”

5 Pre-Operation Checklist

Complete every item in order before each operating session.

- ☐ Vehicle is in a closed, controlled area with no bystanders within 50 ft.
- ☐ Safety driver is seated, belted, and briefed.
- ☐ Mobile hotspot is powered on and broadcasting.
- ☐ Relay server is running on OCI (`python3 relay_server.py`).
- ☐ Relay server shows: `Relay server live on port 4210. Waiting for car heartbeat...`
- ☐ ESP32 is powered and WiFi-connected (Serial Monitor shows `WiFi connected`).
- ☐ Relay server shows: `[OK] CAR CONNECTED! Logged IP: ('x.x.x.x', 4210)`
- ☐ G29 is plugged in and has completed its self-calibration spin.
- ☐ `pygame` is installed on the operator laptop.
- ☐ Vehicle is in **Drive** with foot brake held by the safety driver.

6 Operating the System

6.1 Starting a Session

Once all checklist items are satisfied, run the sender on the operator laptop:

```
python3 g29_to_server_original.py
```

Expected output:

```
Joystick detected: Logitech G29 Driving Force Racing Wheel
Sending UDP to OCI Server at 147.224.143.221:4210
seq=00000 | steer= 0.0000 | throttle=0.0000 | brake=0.0000
seq=00001 | steer= 0.0012 | throttle=0.0000 | brake=0.0000
...
```

The system is now live.

6.2 Control Mapping

G29 Input	Sent Value	Effect
Steering wheel left	-1.0 to 0.0	Steer left (target up to -250°)
Steering wheel right	0.0 to +1.0	Steer right (target up to $+250^\circ$)
Throttle pedal	0.0 to 1.0	Accelerate (open-loop DAC offset)
Brake pedal	0.0 to 1.0	Brake (open-loop DAC offset)

Note

The G29 pedal axes report -1.0 when fully pressed and $+1.0$ when released. The sender inverts this so that 1.0 always means fully pressed.

6.3 Stopping a Session

Press **Ctrl+C** on the operator laptop. There is no automatic watchdog in the firmware — when packets stop arriving, the ESP32 continues targeting the last commanded steering angle, throttle, and brake values. The safety driver must apply the physical foot brake immediately when the remote session ends.

7 How It Works

7.1 Step 1 — Car Registers with the Relay Server

On boot the ESP32 sends a HEARTBEAT UDP packet to the relay server every 2s. This registers the ESP32's public IP/port and keeps the hotspot NAT mapping alive so the server can route packets back.

7.2 Step 2 — Operator Sends Control Commands

`g29_to_server_original.py` polls the G29 at 60 Hz and sends:

CMD; seq; steer; throttle; brake

Field	Range	Description
steer	-1.0... +1.0	-1.0 = full left, +1.0 = full right
throttle	0.0...1.0	0.0 = released, 1.0 = fully pressed
brake	0.0...1.0	0.0 = released, 1.0 = fully pressed

7.3 Step 3 — Relay Server Forwards to the Car

`relay_server.py` receives `CMD` packets from the operator laptop and forwards them verbatim to the last registered `HEARTBEAT` address.

7.4 Step 4 — ESP32 Drives the Actuators

7.4.0 Steering (Closed-Loop PID)

$$\underbrace{\text{steer}}_{-1\dots+1} \times 250 \longrightarrow \text{targetSTA} \xrightarrow{\text{low-pass filter}} \text{smoothTarget} \xrightarrow{\text{PID + CAN feedback}} \text{torque}$$

DAC 2, Ch C = 2.47 V baseline + torque

DAC 2, Ch D = 2.47 V baseline - torque

Actual steering angle is read from CAN ID 0x2 (little-endian `int16`, divide by 10 for degrees) and fed back every loop.

7.4.0 Throttle (Open-Loop)

$$\text{throttle} \times 300 = \text{accelMag} \quad (0\text{--}300)$$

DAC 1, Ch A = 0.37 V + `accelMag`

DAC 1, Ch B = 0.75 V + 2 × `accelMag`

7.4.0 Brake (Open-Loop)

$$\text{brake} \times 900 = \text{brakeMag} \quad (0\text{--}900)$$

DAC 1, Ch C = 3.42 V - `brakeMag`

DAC 1, Ch D = 1.51 V + `brakeMag`

8 Troubleshooting

Symptom	Likely Cause	Fix
No joystick found.	G29 not detected	Unplug/replug G29; wait for calibration spin; rerun.
CAR CONNECTED never appears	ESP32 cannot reach OCI	Confirm hotspot is on; confirm UDP 4210 is open in OCI security list and <code>iptables</code> .
Car does not respond	CMD packets not reaching ESP32	Confirm relay server is running and CAR CONNECTED was printed.
Steering drives to limit and stays	CAN bus not working; no PID feedback	Check MCP2515 wiring; verify 16 MHz crystal on PCB.
MCP2515 Init Failed (Serial Monitor)	SPI wiring or wrong CS pin	Check wiring against Section 3 pin table.
Throttle/brake has no effect	DAC 1 values wrong	Open Serial Monitor; verify <code>accelMag/brakeMag</code> change when pedals are pressed.
Steering oscillates or overshoots	PID gains need tuning	Reduce <code>kp</code> ; increase <code>kd</code> to damp. Retest on stationary vehicle.

9 Multi-Camera Streaming with OBS

The car-side setup supports USB cameras for a live video feed. To broadcast multiple camera angles simultaneously to a video call:

1. Install **OBS Studio** (<https://obsproject.com>).
2. Add each USB camera as a **Video Capture Device** source and arrange them in a single OBS scene (e.g., side-by-side or picture-in-picture).
3. Enable **OBS Virtual Camera** via the Controls panel (*Start Virtual Camera*).
4. In Zoom (or any conferencing app), select **OBS Virtual Camera** as the camera source.

The remote participant sees the full multi-camera OBS scene without screen sharing.